

Sparse Matrix-Vector Multiplication Investigation

Matteo Morellini - 268427

University of Trento

github.com/MatteoMorellini/GPU-Computing-Deliverable-1

matteo.morellini@studenti.unitn.it

Abstract—Sparse matrix-vector multiplication is strongly affected by memory access irregularity and load imbalance. This work compares five CSR-based SpMV implementations on an NVIDIA A30 across ten SuiteSparse matrices. The results show that no custom kernel is uniformly optimal: performance depends mainly on row-length distribution, memory coalescing, and long-row handling. cuSPARSE is the most robust baseline, while CSR-Adaptive is the strongest custom implementation.

I. INTRODUCTION

Sparse matrix-vector multiplication (SpMV), $y = Ax$, is a core operation in iterative solvers and graph workloads. On GPUs, its low arithmetic intensity, irregular accesses to the dense vector, and uneven row lengths make performance strongly dependent on matrix structure and work distribution. This report asks how five direct-CSR implementations respond to these constraints on an NVIDIA A30 across ten SuiteSparse matrices: CSR-Scalar, CSR-Vector, CSR-Adaptive, CSR-Partial-Overlap, and cuSPARSE. The comparison focuses on preprocessing cost, kernel throughput, memory access, and load balance while excluding alternative sparse formats and their conversion overhead.

II. METHODOLOGY

A. Motivations to Exclude Non-CSR Formats

The non-zero pattern induces indirect accesses to x ; irregular column indices can scatter a warp's requests across several memory regions, increasing memory traffic. To mitigate this, domain-specific formats exploit the structural regularity that characterizes matrices from particular fields: graph and web matrices tend to be highly irregular, while matrices from theoretical chemistry often exhibit a block structure imposed by physical and chemical constraints.

Formats such as DIA, BCSR, and ELL [2] were designed with this structure in mind. The downside, however, is their domain specificity: they fail to generalize uniformly across matrix classes. Because of its broad applicability, CSR has become the de facto standard compression format. Converting an existing CSR matrix into one of these specialized formats incurs both runtime and memory overheads [3]. While such overhead can be amortized over many SpMV iterations, on a single computation the conversion time alone vastly exceeds the cost of the multiplication itself [4]. Thus this paper restricts its scope to kernels that operate directly on the CSR format.

B. Variations of CSR Tested

Recall that in CSR format, a sparse matrix A of size $m \times n$ with nnz non-zeros is represented by three arrays: `values` and `col_idx` of length nnz , and `row_ptr` of length $m+1$, where `row_ptr[i]` stores the index in `values` of the first non-zero element of row i . The sequential SpMV computation follows directly from this representation.

For basic CSR-based GPU kernels, the bandwidth-bound nature of SpMV is compounded by irregular, often low-locality accesses to the dense vector and by potential load imbalance on irregular matrices [3]. Subsequent algorithms have therefore sought to preserve the advantages of CSR while mitigating its weaknesses. Two broad strategies appear in the literature:

- 1) Transform the matrix into a modified storage format to improve GPU utilization (e.g., CSR5). Such approaches share the conversion overheads discussed above.
- 2) Improve the SpMV kernel itself to fully exploit GPU hardware while leaving the CSR structure intact.

The second strategy is the one adopted here. Specifically, the following algorithms are evaluated.

CSR-Scalar [1] assigns one thread per row. Adjacent lanes begin at positions given by consecutive `row_ptr` entries, so irregular row lengths produce scattered, often poorly coalesced accesses and uneven work across the warp.

CSR-Vector [1] assigns one warp per row. At each iteration, active lanes read consecutive entries from one row, enabling coalesced warp-level loads; short rows still leave many lanes idle. Formally, if a row contains $k \ll W$ non-zeros, where W is the warp size, then the warp utilization is at most k/W , leading to potential load imbalance within thread blocks.

CSR-Adaptive [4] was designed to address the cases where both previous algorithms degrade. Contiguous rows are grouped dynamically until their combined non-zero count fills at most one thread block, and a single block is then assigned to each such group. As explained in Algorithm 1, two execution paths are selected at runtime based on the number of merged rows:

- **CSR-Stream** is used when multiple rows fit within one block. Each thread loads its assigned non-zero element into shared memory, and a single thread per row performs the reduction. This directly addresses the idle-thread problem of CSR-Vector on short rows.
- A block-level variant of CSR-Vector is used when a single row contains more non-zeros than threads in a block, dedicating the entire block to that row.

Algorithm 1 CSR-Adaptive

```

1: CPU: partition rows into blocks s.t.  $\text{nnz/block} \leq \text{BLOCK-SIZE}$ 
    $O(m)$ 
2: for each block  $g$  in parallel do
3:   if  $|\text{rows in } g| \geq \text{SMALLVALUE}$  then
4:     CSR-Stream: stream into shared mem; scalar reduction per row
5:   else
6:     CSR-Vector: one warp per row; parallel reduction
7:   end if
8: end for

```

Algorithm 2 Two-stage pipeline in CSR-Partial-Overlap

```

1: for each batch  $b$  do
2:   Produce: async_copy(valsb, smem[b mod 2])
3:   Consume: compute smem[(b-1) mod 2] after copy
4: end for

```

The row-block partitioning required by CSR-Adaptive is generated on the CPU side before kernel launch. Its complexity is $O(m)$, far cheaper than converting CSR into a format.

CSR-Partial-Overlap [5] exploits the asynchronous memory copy instruction introduced by the NVIDIA Ampere architecture. The `memcpy_async` instruction moves data from global to shared memory without intermediate register staging, allowing data transfer to overlap computation. This enables the two-stage pipeline shown in Algorithm 2.

Load balancing is achieved through a dynamic batch partition that ensures no row is split across two batches, thereby eliminating the partial-sum fixup step that would otherwise require additional global memory accesses. Thread utilization within each batch is further improved by dynamically selecting the number of threads assigned to each row based on the average non-zero count of that batch.

cuSPARSE is included as an optimized baseline.

III. EXPERIMENTAL SETUP

A. Hardware and Implementation Setup

Experiments run on an NVIDIA A30 with CUDA 11.8.0, using Float32 inputs generated with a fixed random seed.

B. Dataset

Ten matrices from the SuiteSparse Matrix Collection were selected to cover diverse sparsity regimes and application domains, as summarized in Table I. Symmetric matrices are stored in expanded form to reflect raw CSR performance on the full non-zero structure.

C. Validation and Metrics

All GPU outputs are validated against an eight-thread OpenMP CPU CSR reference that accumulates each row in double precision, using a relative tolerance of $10^{-3} \max(1, |y_i^{\text{cpu}}|)$. Kernel time is measured with CUDA events over 100 repetitions after 5 warmup runs. Throughput is reported as $2 \cdot \text{nnz} / t$ GFLOP/s.

TABLE I

SPARSE MATRIX STATISTICS ON PER-ROW NON-ZERO COUNT. SYMMETRIC MATRICES ARE STORED IN EXPANDED FORM.

Matrix	Dim.	nnz	med	avg	std	max
ASIC_680ks	683K	2.3M	2	3.4	4.3	210
FullChip	3.0M	26.6M	6	8.9	1806.8	2.3M
Rucci1*	2.0M	7.8M	4	3.9	0.3	4
Si41Ge41H72	186K	15.0M	37	80.9	127.0	662
bone010	987K	71.7M	81	72.6	15.8	81
boyd2	466K	1.5M	3	3.2	194.9	93K
eu-2005	863K	19.2M	16	22.3	29.3	7.0K
ldoor	952K	46.5M	49	48.9	12.0	77
rajat31	4.7M	20.3M	5	4.3	1.1	1.3K
webbase-1M	1.0M	3.1M	2	3.1	25.4	4.7K

* Rucci1 is rectangular ($2.0\text{M} \times 110\text{K}$); Dim. reports its row count.

TABLE II

ONE-TIME SETUP COST AND KERNEL TIME. PARSING IS EXCLUDED.

Matrix	Conv. + H2D (ms)					Kernel (ms)				
	Scal.	Vec.	Adap.	Part.	cuSP.	Scal.	Vec.	Adap.	Part.	cuSP.
bone010	761	696	728	750	721	1.39	1.36	1.22	1.34	1.00
webbase-1M	46	42	48	52	45	0.50	0.54	0.08	0.26	0.09
ASIC_680ks	32	30	34	37	31	0.07	0.37	0.06	0.14	0.08
ldoor	528	482	621	524	487	1.02	0.99	0.78	1.10	0.67
rajat31	250	234	289	270	237	0.34	3.01	0.38	0.53	0.42
Rucci1	128	119	141	180	121	0.12	1.37	0.13	0.22	0.18
FullChip	312	311	438	325	320	199.3	10.1	9.4	8.2	0.48
Si41Ge41H72	160	160	226	163	167	0.45	0.25	0.26	0.41	0.24
boyd2	15	15	19	17	16	6.38	0.60	0.41	0.36	0.05
eu-2005	234	234	325	247	249	0.99	0.73	0.36	0.59	0.31

IV. RESULTS

A. Preprocessing and Transfer Cost

Table II reports one-time COO-to-CSR conversion and host-to-device transfer alongside kernel time; parsing is excluded, and kernel timing excludes all setup. CSR-Adaptive and CSR-Partial-Overlap additionally include CPU row-block construction and transfer of block metadata, although this cost is masked by the larger common conversion cost. Setup can be amortized when the same resident matrix and partition metadata are reused across repeated SpMV calls.

B. Kernel Performance

Table III reports the throughput of every kernel on every matrix. Run-to-run variability is generally small; two kernel-matrix pairs exceed 1 GFLOP/s in standard deviation, with a maximum of 2.2 GFLOP/s for CSR-Vector on *rajat31*.

cuSPARSE dominates on 6 of 10 matrices and peaks at 143.6 GFLOP/s on *bone010*. Two custom kernels overtake it on the remaining four: CSR-Adaptive on *ASIC_680ks* and *webbase-1M*, while CSR-Scalar on *Rucci1* and *rajat31*. On aggregate, CSR-Adaptive is the strongest custom kernel (58.9 GFLOP/s geomean), followed by CSR-Partial-Overlap (41.2), CSR-Vector (22.9), and CSR-Scalar (22.6).

Two severe performance collapses stand out. CSR-Scalar drops to 0.3 and 0.5 GFLOP/s on *FullChip* and *boyd2*, respectively, the two matrices with the largest single-row *nnz* in the dataset (Table I). CSR-Adaptive collapses on the same pair, reaching only 5.6 and 7.3 GFLOP/s. CSR-Vector remains below 15 GFLOP/s on six matrices: the four with

TABLE III
SpMV THROUGHPUT ACROSS THE MATRIX PORTFOLIO (GFLOP/s).

Kernel	bone010	webbase-1M	ASIC_680ks	ldoor	rajat31	Ruccil	FullChip	Si41Ge41H72	boyd2	eu-2005	Geo.
CSR-Scalar	95.2	12.6	64.9	90.4 [†]	119.8	130.5	0.3	66.1	0.5	38.5	22.6
CSR-Vector	104.9 [†]	11.6	12.7	90.9 [†]	14.6	11.3	5.3	121.1	5.0	52.7	22.9
CSR-Adaptive	117.5	79.7	73.4	118.2	105.5	116.0	5.6	116.1	7.3	107.4	58.9
CSR-Partial-Overlap	97.1 [†]	27.5	44.2	83.8 [†]	77.3	72.4	6.6	72.9	8.2	65.3	41.2
cuSPARSE	143.6[†]	65.8	59.5	138.8[†]	96.5	85.7	111.6	127.5	56.8	124.5	95.8

[†] Float32 result that failed the 10^{-3} relative tolerance. **Bold** marks the best kernel per matrix; *Geo.* is the geometric mean across the portfolio.

the shortest rows (ASIC_680ks, webbase-1M, Ruccil, rajat31, all with median per-row $nnz \leq 5$), plus FullChip and boyd2. CSR-Partial-Overlap shows the narrowest performance spread of the four custom kernels (14.8 $\times$ between its minimum and maximum, against 21.2 $\times$ for CSR-Adaptive, 24.1 $\times$ for CSR-Vector, and 486.2 $\times$ for CSR-Scalar), though it never matches the peak throughput of the other three. cuSPARSE, by contrast, spans only 2.5 $\times$, remaining above 56 GFLOP/s across the entire portfolio.

The daggered results for bone010 and ldooor arise from cancellation in some row-wise dot products. Different parallel reduction orders produce small Float32 rounding differences that can exceed the relative tolerance when the reference value is near zero.

V. DISCUSSION

Each entry in Table III reflects a specific interaction between a kernel’s memory-access strategy, its thread-work distribution, and the structural properties of the matrix.

A. Memory Coalescing and Access Patterns

Given SpMV’s low arithmetic intensity, the results are explained mainly by memory-access efficiency and thread utilization rather than by raw floating-point throughput.

CSR-Scalar assigns one thread per row, so within a warp the i -th thread begins reading at `values[row_ptr[i]]`. For irregular row lengths, active lanes may touch several separated memory regions, requiring multiple transactions rather than the compact transactions of warp-contiguous accesses. The gather `x[col_idx[j]]` adds a second source of irregularity through non-sequential column indices. Despite this, CSR-Scalar attains 77.8% of peak on Ruccil (maximum $nnz = 4$). With every row containing four elements, lane i accesses approximately `values[4i+k]` at iteration k . This regular stride-four pattern is not ideally coalesced, but uniform row lengths provide perfectly balanced lane execution and favorable memory behavior. A similar, though less uniform, short-row pattern helps explain the 69.1% peak on rajat31 (median $nnz = 5$). This uniform-short-row regime is the one structural niche where CSR-Scalar is competitive even against cuSPARSE.

CSR-Vector assigns one warp per row. At each iteration, active lanes access consecutive values and column indices within one row, enabling coalesced warp-level loads; short rows underfill these transactions. The irregular `x[col_idx]`

gather and lane underutilisation then limit effective bandwidth. Where rows are long enough to keep all 32 lanes busy (Si41Ge41H72, median $nnz = 37$), CSR-Vector reaches 53.1% of peak, essentially matching CSR-Adaptive and approaching cuSPARSE. Where rows are short, warp underutilisation collapses bandwidth utilisation to under 8% (ASIC_680ks, Ruccil, rajat31).

CSR-Adaptive cooperatively loads up to `NNZ_PER_BLOCK = 1024` consecutive non-zeros. At each iteration, every warp reads 32 adjacent entries, while each lane advances by `THREADS_PER_BLOCK = 256` entries between iterations, yielding warp-level coalesced transactions. This access pattern is consistent with the observed 50–57% bandwidth utilisation across varied row-length distributions.

CSR-Partial-Overlap uses similar warp-contiguous loads and pipelines global-to-shared copies with `memcpy_async`, overlapping transfer of the next batch with computation on the current batch. However, the kernel’s computation path caps `vector_size` at 32, meaning at most one warp is ever assigned to a single row. For matrices with very long rows, such as FullChip (max $nnz = 2.3 \times 10^6$), those 32 threads must iterate $\sim 71,875$ times per row while the rest of the block remains idle, a ceiling identical in effect to the single-warp fallback of the paper version of CSR-Adaptive. Rows that exceed `max_batch_size` are routed to the dedicated long-row kernel, which allocates a full block per row, but this path incurs additional kernel-launch overhead and does not benefit from the `memcpy_async` pipeline. Moreover for extremely-long rows, as in FullChip, this is insufficient since other methods use multiple thread blocks.

B. Load Imbalance and Warp Utilisation

Even when memory accesses are coalesced, a kernel wastes bandwidth if threads within a warp complete at different times. Two failure modes appear prominently.

Long-row serialisation severely degrades CSR-Scalar, CSR-Partial-Overlap, and CSR-Adaptive on FullChip and boyd2. On FullChip’s 2.3×10^6 - nnz row, the CSR-Scalar thread assigned to it executes alone while 31 warp members wait, reducing HBM utilisation to 0.1% of peak. The original CSR-Adaptive routes the row to its single-warp CSR-Vector fallback (no row may exceed `NNZ_PER_BLOCK` without a dedicated long-row handler), reaching only 2.8% of peak.

Short-row warp underutilisation defeats CSR-Vector on the four matrices with median $nnz \leq 5$. On ASIC_680ks (median $nnz = 2$), a median-length row activates only 2

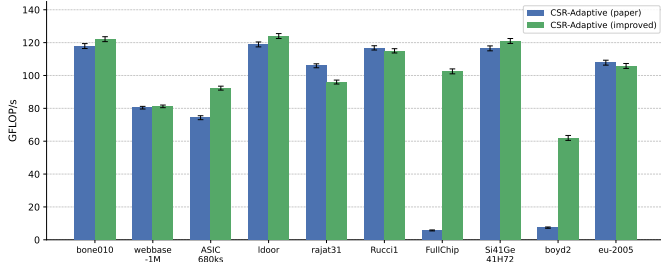


Fig. 1. Throughput comparison of the paper-faithful and improved CSR-Adaptive implementations

lanes out of 32, for active-lane utilisation of 6.3%, driving effective bandwidth to 7.8% of peak. CSR-Adaptive’s Stream path mitigates this by grouping multiple short rows into one block, improving block utilisation by packing several rows into a cooperative reduction.

C. Ablation: CSR-Adaptive Paper vs. Improved

Fig. 1 compares the faithful implementation [4] against the improved version implemented in this work. The paper version’s drop on FullChip and boyd2 traces to an omission in the block builder: any row whose *nnz* exceeds *NNZ_PER_BLOCK* is routed to the single-warp CSR-Vector fallback discussed in Section V-B. The improved version introduces a dedicated *csr_longrow_kernel* that splits such rows into 1024-element chunks, each handled by an independent thread block with partial sums accumulated via *atomicAdd*. Speedups are 18× on FullChip (5.7 → 102.3 GFLOP/s) and 8.4× on boyd2 (7.4 → 62.1 GFLOP/s); other matrices show only minor drops from the block-size tradeoff.

D. Kernel-Level Profiling with Nsight Systems

Per-kernel timings on FullChip, whose 2.3×10^6 -nnz long row stresses every long-row handler, corroborate the load-imbalance analysis of Sections V-B and V-C.

In CSR-Partial-Overlap, the long-row kernel consumes 7.2 ms (93.5% of total time) while the pipelined batch kernel completes in 505 μ s, explaining why prefetching only pays off on matrices whose maximum row fits within a batch. Paper-faithful CSR-Adaptive is worse: the CSR-Vector fallback spends 8.0 ms on the long row against 360 μ s of CSR-Stream for the remaining $\sim 26.6 \times 10^6$ non-zeros, so one row accounts for $\approx 96\%$ of kernel time. The improved version cuts the long-row contribution to 80 μ s via block-level parallelisation (400 μ s Stream); the 480 μ s aggregate is a 17.4× reduction over the 8.36 ms paper baseline, matching the 18× end-to-end speedup in Fig. 1 and confirming block-level parallelisation as the decisive factor on extreme row-length outliers.

E. Nsight Compute Comparison: CSR-Scalar vs. CSR-Vector

Table IV confirms that the two kernels fail for opposite reasons depending on matrix structure. On Rucci1, uniform four-element rows give CSR-Scalar balanced lane execution and a regular strided access pattern, reaching 683.3 GB/s

TABLE IV
CSR-SCALAR AND CSR-VECTOR PROFILING METRICS.

Matrix	Kernel	BW (GB/s)	DRAM (%)	L1 (%)	L2 (%)	Thr/warp
ASIC_680ks	Scalar	307.9	33.0	78.7	37.0	14.3
	Vector	54.6	5.9	74.7	62.4	16.3
FullChip	Scalar	1.1	0.1	82.0	49.7	11.5
	Vector	23.1	2.5	64.6	56.4	18.1
Rucci1	Scalar	683.3	73.3	75.2	38.6	28.7
	Vector	57.4	6.2	78.5	66.8	16.5

BW: DRAM throughput; DRAM%: fraction of device peak; L1/L2: cache hit rates; Thr/warp: active threads per warp.

despite the absence of ideal warp-contiguous accesses. CSR-Vector wastes 28 of 32 lanes per row, collapsing to 57.4 GB/s and dropping GFLOP/s from 130.5 to 11.3. On ASIC_680ks the picture is similar but weaker: CSR-Scalar still outperforms (307.9 vs. 54.6 GB/s) because its short rows often keep simultaneous CSR accesses within a relatively compact address range, while CSR-Vector starves its warps. On FullChip the bottleneck shifts entirely to the 2.3M-non-zero row: CSR-Scalar serialises it onto one thread, yielding 1.1 GB/s and a 272 ms runtime (the 82% L1 hit rate does not offset the serialization or imply overall memory efficiency); CSR-Vector distributes the row across 32 threads ($22\times$ faster, 12.4 ms) but at 2.5% DRAM utilisation remains far from memory-bound.

VI. CONCLUSION

The results confirm that CSR SpMV performance is determined less by raw floating-point throughput than by memory access efficiency and load balance. cuSPARSE is the most robust implementation overall, but custom kernels remain competitive on specific sparsity regimes. CSR-Scalar benefits from uniformly short rows, CSR-Vector from rows long enough to fill a warp, and CSR-Adaptive from grouping rows to reduce underutilisation. The main limitation is the treatment of very long rows, where splitting work among blocks is key to avoid serialization. Finally, the custom kernels do not directly regularize the irregular gather $x[\text{col_idx}[j]]$; techniques that could reduce its randomness were left out of scope.

REFERENCES

- [1] N. Bell and M. Garland, "Implementing sparse matrix-vector multiplication on throughput-oriented processors," in *Proc. Conf. High Performance Computing Networking, Storage and Analysis (SC '09)*, 2009, pp. 1–11.
- [2] J. Gao, B. Liu, W. Ji, and H. Huang, "A systematic literature survey of sparse matrix-vector multiplication," 2024, arXiv:2404.06047. [Online]. Available: <https://arxiv.org/abs/2404.06047>
- [3] G. Chu, Y. He, L. Dong, Z. Ding, D. Chen, H. Bai, X. Wang, and C. Hu, "Efficient algorithm design of optimizing SpMV on GPU," in *Proc. 32nd Int. Symp. High-Performance Parallel and Distributed Computing (HPDC '23)*, 2023, pp. 115–128. DOI: 10.1145/3588195.3593002.
- [4] J. L. Greathouse and M. Daga, "Efficient sparse matrix-vector multiplication on GPUs using the CSR storage format," in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC '14)*, IEEE, 2014, pp. 769–780.
- [5] G. Zeng and Y. Zou, "Leveraging memory copy overlap for efficient sparse matrix-vector multiplication on GPUs," *Electronics*, vol. 12, no. 17, p. 3687, 2023. DOI: 10.3390/electronics12173687.